Support vector machines

Fraida Fund

Contents

Maximal margin classifier . . . 2
- Binary classification problem . . . 2
- Linear separability . . . 2
- Separating hyperplane (1) . . . 2
- Separating hyperplane (2) . . . 2
- Using the hyperplane to classify . . . 2
- Which separating hyperplane is best? . . . 3
- Margin . . . 3
- Classifier that maximizes the margin . . . 4
- Support vectors . . . 4
- Constructing the maximal margin classifier . . . 4
- Constructing the maximal margin classifier (1) . . . 5
- Constructing the maximal margin classifier (2) . . . 6
- Problems with MM classifier (1) . . . 6
- Problems with MM classifier (2) . . . 6

Support vector classifier . . . 7
- Basic idea . . . 7
- Constructing the support vector classifier . . . 7
- Support vector . . . 8
- Illustration of effect of K . . . 8
- K controls bias-variance tradeoff . . . 8

Solution . . . 9
- Fitting the support vector classifier . . . 9
- Dual formulation (we won't derive it) . . . 9
- Solution (we won't derive it) . . . 10
- Why this form matters . . . 10
- Loss function . . . 10
- Compared to logistic regression . . . 10
- Relationship between SVM and other models . . . 11
- Correlation interpretation (1) . . . 11
- Correlation interpretation (2) . . . 11

Maximal margin classifier

Binary classification problem

- N training samples, each with p features $\mathbf{x}_1, \ldots, \mathbf{x}_N \in \mathbb{R}^p$
- Class labels $y_1, \ldots, y_N \in \{-1, 1\}$

Linear separability

The problem is **perfectly linearly separable** if there exists a **separating hyperplane** H_i such that

- all $\mathbf{x} \in C_i$ lie on its positive side, and
- all $\mathbf{x} \in C_j, j \neq i$ lie on its negative side.

In the binary classification case: The data are linearly separable if we can find a hyperplane that places all $y = 1$ points on one side and all $y = -1$ on the other.

Separating hyperplane (1)

The separating hyperplane has the property that for all $i = 1, \ldots, N$,

$$w_0 + \sum_{j=1}^{p} w_j x_{ij} > 0 \text{ if } y_i = 1$$

$$w_0 + \sum_{j=1}^{p} w_j x_{ij} < 0 \text{ if } y_i = -1$$

Separating hyperplane (2)

Equivalently:

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) > 0 \tag{1}$$

(we mention this compact form because we will use it in our formulation of the classifier.)

Using the hyperplane to classify

Then, we can classify a new sample $\mathbf{x}$ using the sign of

$$z = w_0 + \sum_{j=1}^{p} w_j x_j$$

The score z is real-valued; the final class is

$$\hat{y} = \text{sign}(z).$$

We can use $|z|$ to measure distance from the hyperplane. Larger $|z|$ means the sample is farther from the hyperplane.

Which separating hyperplane is best?

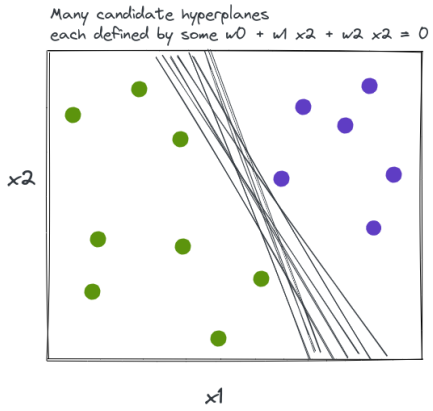


Figure 1: If the data is linearly separable, there are many separating hyperplanes.

We said there will be infinitely many separating hyperplanes, if there is one:

- Previously, with the logistic regression, we found the maximum likelihood classifier: the hyperplane that maximizes the probability of these particular observations.
- This time, we'll find a different one.

Margin

For any "candidate" hyperplane,

- Compute distance from each sample to separating hyperplane.
- Smallest distance among all samples is called the **margin**.

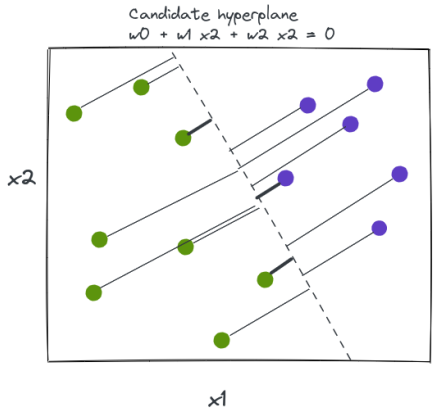


Figure 2: For this hyperplane, bold lines show the smallest distance (tie among several samples).

Classifier that maximizes the margin

- Among all separating hyperplanes, choose the one with the largest margin!
- Find the widest "slab" we can fit between the two classes; use the midline of this "slab".

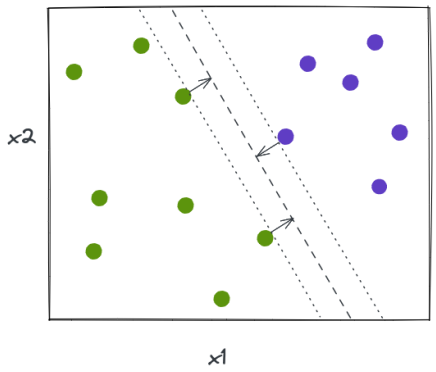


Figure 3: Maximal margin classifier. Width of the "slab" is 2x the margin.

Support vectors

- Points that lie on the border of maximal margin hyperplane are **support vectors**
- They "support" the maximal margin hyperplane: if these points move, then the maximal margin hyperplane moves
- Maximal margin hyperplane is not affected by movement of any other point, as long as it doesn't cross borders!

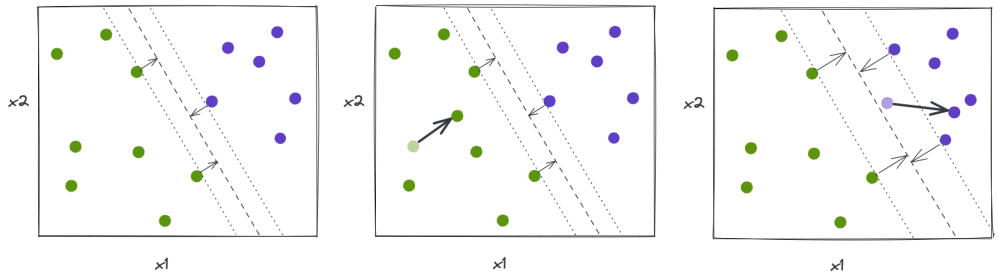


Figure 4: Maximal margin classifier (left) is not affected by movement of a point that is not a support vector (middle) but the hyperplane and/or margin are affected by movement of a support vector (right).

Constructing the maximal margin classifier

To construct this classifier, we will set up a *constrained optimization* problem with:

- an objective
- one or more constraints to satisfy

What should the objective/constraints be in this scenario?

Constructing the maximal margin classifier (1)

$$\underset{\mathbf{w},\gamma}{\text{maximize}}\ \gamma \tag{2}$$

$$\text{subject to: } \sum_{j=1}^{p} w_j^2 = 1 \tag{3}$$

$$\text{and } y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma, \forall i \tag{4}$$

The constraint

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma, \forall i$$

guarantees that each observation is on the correct side of the hyperplane *and* on the correct side of the margin, if margin γ is positive. (This is analogous to Equation 1, but we have added a margin.)

The constraint

$$\text{and } \sum_{j=1}^{p} w_j^2 = 1$$

is not really a constraint: if a separating hyperplane is defined by $w_0 + \sum_{j=1}^{p} w_j x_{ij} = 0$, then for any $k \neq 0$, $k \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) = 0$ is also a separating hyperplane.

This "constraint" just scales $\mathbf{w}$ so that distance from ith sample to the hyperplane is given by $y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right)$. This is what make the previous constraint meaningful!

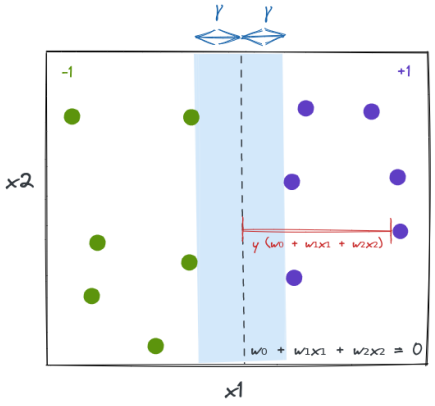


Figure 5: Maximal margin classifier.

Constructing the maximal margin classifier (2)

The constraints ensure that

- Each observation is on the correct side of the hyperplane, and
- at least γ away from the hyperplane

and γ is maximized.

Problems with MM classifier (1)

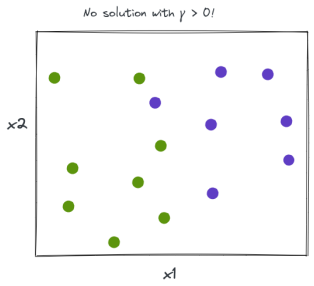


Figure 6: When data is not linearly separable, optimization problem has no solution with $\gamma > 0$.

Problems with MM classifier (2)

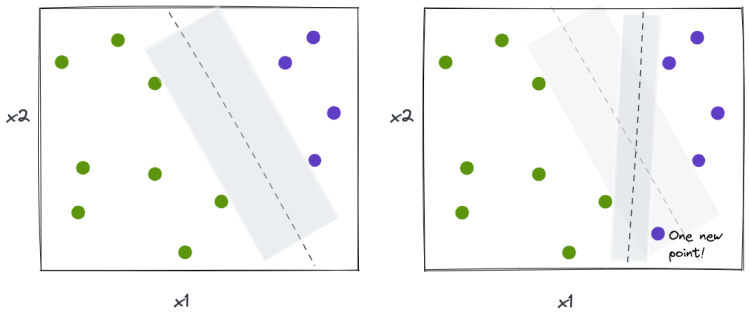


Figure 7: The classifier is not robust - one new observation can dramatically shift the hyperplane.

Support vector classifier

Basic idea

- Generalization of MM classifier to non-separable case
- Use a hyperplane that *almost* separates the data
- "Soft margin"

Constructing the support vector classifier

$$\underset{\mathbf{w},\boldsymbol{\epsilon},\gamma}{\text{maximize}}\ \gamma \tag{5}$$

$$\text{subject to: } \sum_{j=1}^{p} w_j^2 = 1 \tag{6}$$

$$y_i \left(w_0 + \sum_{j=1}^{p} w_j x_{ij} \right) \geq \gamma(1 - \epsilon_i), \quad \forall i \tag{7}$$

$$\epsilon_i \geq 0, \quad \forall i, \quad \sum_{i=1}^{N} \epsilon_i \leq K \tag{8}$$

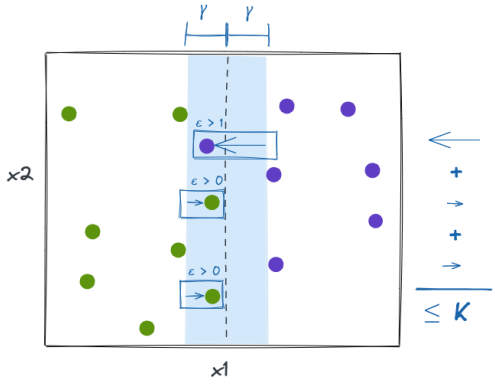


Figure 8: Support vector classifier. Note: the blue arrows show $y_i \gamma \epsilon_i$.

K is a non-negative tuning parameter.

Slack variable ϵ_i determines where a point lies:

- If $\epsilon_i = 0$, point is on the correct side of margin
- If $\epsilon_i > 0$, point has *violated* the margin (wrong side of margin)
- If $\epsilon_i > 1$, point is on wrong side of hyperplane and is misclassified

K is the **budget** that determines the number and severity of margin violations we will tolerate.

- $K = 0 \rightarrow$ same as MM classifier
- $K > 0$, no more than K observations may be on wrong side of hyperplane
- As K increases, more violations allowed, margin can be wider

Support vector

For a support vector classifier, the only points that affect the classifier are:

- Points that lie on the margin boundary
- Points that violate margin

These are the *support vectors*.

Illustration of effect of K

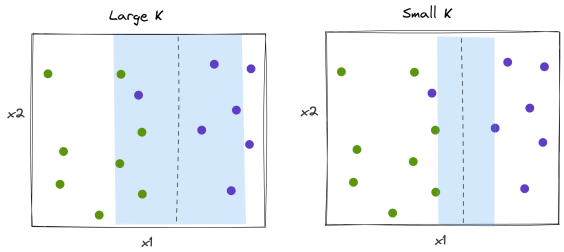


Figure 9: The margin shrinks as K decreases.

K controls bias-variance tradeoff

- Wide margin (K large): many support vectors, low variance, high bias.
- Narrow margin (K small): few support vectors, high variance, low bias.

Terminology note: In ISL and in the first part of these notes, meaning of constant is opposite its meaning in Python `sklearn`:

- ISL and these notes: Large K, wide margin.
- Python `sklearn`: Large C, small margin.

Solution

Fitting the support vector classifier

$$\begin{aligned} \underset{\mathbf{w},\boldsymbol{\epsilon}}{\text{minimize}} \quad & \frac{1}{2}\sum_{j=1}^{p} w_j^2 + C\sum_{i=1}^{N}\epsilon_i \\ \text{subject to} \quad & y_i(w_0 + \sum_{j=1}^{p} w_j x_{ij}) \geq 1 - \epsilon_i, \quad \forall i \\ & \epsilon_i \geq 0, \quad \forall i \end{aligned}$$

- $\frac{1}{2}\sum_j w_j^2$: small when the margin is **wide**
- $C\sum_i \epsilon_i$: penalty for **margin violations**; C sets the tradeoff
- Constraints: each sample is on the correct side of its margin boundary, up to its slack ϵ_i

Here ϵ_i is the slack for sample i: how far it is on the wrong side of its margin boundary (0 if it is on the correct side).

Large C: violations are expensive, so the margin gets narrower. Small C: violations are cheap, so the margin gets wider. (This is the `sklearn` meaning of C; it is the opposite direction from the budget K.)

This is the same problem as the previous formulation, rescaled so that the margin boundaries are at ± 1 (then the margin is $1/\|\mathbf{w}\|$, so maximizing the margin means minimizing $\sum_j w_j^2$), and with the budget K moved into the objective as a penalty weighted by C. We won't derive how it is solved (the derivation uses Lagrange multipliers), but we will look at the form of the solution.

Dual formulation (we won't derive it)

Using Lagrange multipliers, the problem can be rewritten in an equivalent *dual* form, with one variable α_i per training sample:

$$\begin{aligned} \underset{\alpha_i \geq 0, \forall i}{\text{maximize}} \quad & \sum_{i=1}^{N}\alpha_i - \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N}\alpha_i\alpha_j y_i y_j \mathbf{x}_i^T\mathbf{x}_j \\ \text{subject to} \quad & \sum_{i=1}^{N}\alpha_i y_i = 0 \\ & 0 \leq \alpha_i \leq C, \quad \forall i \end{aligned}$$

- The data only appears through inner products $\mathbf{x}_i^T\mathbf{x}_j$ between pairs of samples.
- α_i is non-zero only for support vectors.

The solver finds the α_i values.

Solution (we won't derive it)

The solver gives us a number $\alpha_i \geq 0$ for every training sample, and the coefficients are

$$\mathbf{w} = \sum_{i=1}^{N} \alpha_i \, y_i \, \mathbf{x}_i$$

- $\alpha_i = 0$ for every sample that is not a support vector
- so only the support vectors contribute to the sum
- w_0 can be found from any support vector

Why this form matters

In the dual formulation, the training data only appears through **inner products** $\mathbf{x}_i^T \mathbf{x}_j$ between pairs of samples.

This is what makes the **kernel trick** possible, which we'll discuss next.

Loss function

This problem is equivalent to minimizing hinge loss:

$$\underset{\mathbf{w}}{\text{minimize}} \left(\sum_{i=1}^{N} \max\left(0,\, 1 - y_i z_i\right) + \frac{1}{C} \sum_{j=1}^{p} w_j^2 \right)$$

where $z_i = w_0 + \sum_{j=1}^{p} w_j x_{ij}$.

For a labeled observation $(\mathbf{x}_i, y_i)$ with $y_i \in \{-1, 1\}$, let

$$z_i = w_0 + \sum_{j=1}^{p} w_j x_{ij}$$

The hinge loss for this observation is $\max\left(0,\, 1 - y_i z_i\right)$.

Value of $y_i z_i$	Interpretation	Hinge loss: $\max(0,\, 1 - y_i z_i)$
> 1	Correct and outside the margin	0
$= 1$	Right on the margin	0
Between 0 and 1	Correct but inside the margin	$1 - y_i z_i$ (greater than 0)
≤ 0	Misclassified	$1 - y_i z_i$ (greater than 0)

Hinge loss penalizes points only when they are inside the margin or misclassified!

Compared to logistic regression

- **Hinge loss**: zero for points that are correct and outside margin.
- **Logistic regression loss**: small but non-zero loss for points far from separating hyperplane.

Relationship between SVM and other models

- Like a logistic regression - linear classifier, separating hyperplane is $w_0 + \sum_{j=1}^{p} w_j x_{ij} = 0$
- Like a weighted KNN - the decision score is a weighted sum of support-vector labels, with weights proportional to the similarity between the test sample and each support vector.

Correlation interpretation (1)

Given a new sample $\mathbf{x}$ to classify, compute

$$\begin{aligned} z(\mathbf{x}) &= w_0 + \sum_{j=1}^{p} w_j x_j \\ &= w_0 + \sum_{i=1}^{N} \alpha_i y_i \left(\sum_{j=1}^{p} x_{ij} x_j \right) \\ &= w_0 + \sum_{i=1}^{N} \alpha_i y_i \left(\mathbf{x}_i^\top \mathbf{x} \right) \end{aligned}$$

Measures inner product (a kind of "correlation") between new sample and each support vector.

Correlation interpretation (2)

Final class prediction (assuming $-1, 1$ labels):

$$\widehat{y}(\mathbf{x}) = \text{sign}(z(\mathbf{x}))$$

The score $z(\mathbf{x})$ is a weighted sum of support-vector labels; $\widehat{y}(\mathbf{x})$ is its sign.